

Which Characters Need Context?

Which Characters Need Context?

Decomposing Context Dependence into Trajectory and Magnitude via Per-Character Profiling

Author: [Anonymous]

Affiliation: [Anonymous]

Date: 2026-08-21

Version: Manuscript experiment — per-corpus coverage-probe $k_{\max}$ ($k \leq 7$ NL1, $k \leq 8$ NL2, $k \leq 10$ Code1), Laplace smoothing, cluster-robust SEs

Repository: [Anonymous]

Zenodo DOI: [Anonymous]

Abstract

We introduce a per-character context-gain profiling framework that decomposes context dependence into two distinct estimands: *trajectory*, the rate at which predictive benefit accumulates with additional context, and *magnitude*, the average predictive benefit over the reliable context range. These quantities are partially decoupled by construction and can differ in ways that aggregate perplexity obscures. The framework is reusable for any corpus via a standalone profiling tool.

Applying the framework to natural language and source code, the framework reveals a robust structural-trajectory effect in natural-language prose: in *Pride and Prejudice*, structural characters have significantly steeper trajectories than lexical characters after controlling for frequency, with a second nominally significant cross-corpus result in Reuters. Shakespeare is directionally consistent but weaker. Code corpora serve as contrast cases with no significant trajectory effect.

Across the corpora studied, the framework reveals corpus-specific context sensitivity at character granularity: different character types saturate at different context lengths, and the reliable context range must be measured per corpus rather than inferred from corpus size alone. A coverage diagnostic makes this sparsity limitation explicit and turns it into a reported part of the analysis.

More broadly, the framework provides a reproducible baseline for studying target-specific context requirements and for testing whether similar saturation patterns arise in neural language models or adaptive context systems.

1. Introduction

The history of language modelling begins at the character level. Shannon (1951) estimated the entropy of English by asking human subjects to predict successive letters. Modern large language models report aggregate perplexity, which compresses all character and symbol types into a single number. That compression hides a question worth asking directly: do all individual characters benefit equally from longer context, or does the marginal predictive value of additional context differ across character types?

Prior work addresses related questions from different angles. Shannon entropy and perplexity characterise context dependence in aggregate. Variable-order Markov models and PPM select context length conditioned on the observed history — what appears *before* the prediction target. Neither asks the complementary question:

conditioned on a specific target character, how much does context of length k reduce its surprisal, and does that benefit differ by character type? This per-target decomposition, with frequency control and corpus-specific coverage diagnostics, is the gap the present work addresses (Background, Section 2).

We introduce a per-character context-gain profiling framework that operationalises this measurement. For each character in a corpus we compute $CG_x(k; D)$ — the reduction in mean surprisal from a k -character context relative to a bigram baseline — then decompose the resulting curve into *trajectory* (how steeply gain accumulates per unit log-context) and *magnitude* (the mean gain over the reliable context range). Classifying characters after measurement, not before, avoids the frequency confounds that arise from comparing singleton punctuation symbols against broad alphabetic aggregates. A per-symbol coverage diagnostic gates which observations enter the regression, making the reliable context range explicit and corpus-specific rather than silently assumed.

Applied to natural-language prose and source code, the framework reveals a corpus-dependent dissociation. In the natural-language corpora studied, structural characters have steeper trajectories than lexical characters after controlling for frequency. Across the code corpora studied, no significant trajectory difference emerges. In canonical Python, structural characters additionally show an exploratory Mean Context Gain advantage. Trajectory and magnitude can separate in ways that aggregate perplexity cannot expose.

Contributions:

1. A **reusable per-character context profiling framework and tool** — `character_context_profile.py` takes any corpus file, estimates $S_x(k; D)$ and $CG_x(k; D)$ per individual character, and reports context-gain trajectories with per-symbol coverage diagnostics identifying the reliable context range. A coverage-based reliability diagnostic turns n -gram sparsity from a silent artifact into a reported finding.
2. A **structural-trajectory effect in natural-language prose** — in *Pride and Prejudice*, structural characters have a robustly steeper trajectory than lexical characters after controlling for frequency ($\beta_3=+1.131$, cluster-robust $p=0.002$, permutation $p=0.001$), stable across coverage thresholds, functional forms, five text splits, and lexical definition variants. Reuters provides a second nominally significant cross-corpus result ($\beta_3=+0.712$, $p=0.044$), confirmed by LOO robustness and nominally significant Mean Context Gain.
3. A **code contrast case** — in the Python corpus studied, no significant trajectory effect is observed ($\beta_3=-0.024$, $p=0.930$); structural characters show an exploratory Mean Context Gain advantage. Code corpora serve as a contrast, not a second headline result.
4. A **trajectory/magnitude dissociation** — trajectory (rate of accumulation) and magnitude (mean gain over the reliable range) are partially decoupled by construction and dissociate in the corpora studied: in canonical Python, structural characters achieve higher mean CG than lexical characters without doing so through a steeper accumulation rate. This dissociation is hidden by aggregate perplexity.
5. **Cross-corpus and robustness evidence** — all five NL corpora show positive β_3 ; the NL/code domain partition achieves the maximum mean β_3 separation across all $C(8,5)=56$ assignments (exact permutation $p=0.018$). Four robustness checks confirm NL2: LOO regression, KT smoothing, five text splits, lexical definition sensitivity.

2. Background and Related Work

Prior work measures context dependence either in aggregate — Shannon entropy over all characters, perplexity over all tokens — or studies punctuation's role inside neural attention weights. Variable-order Markov models (VOM, PPM) select context length conditioned on the observed history; the present work instead conditions on the target character, measuring how much each target benefits from longer context. No prior work combines per-character decomposition, frequency-controlled regression, and corpus comparison across prose and source code.

Character-level entropy. Shannon (1951) measured $H(k)$ for English prose as a whole. Brown et al. (1992) estimated entropy decay with context. Scheibner et al. (2025) showed character-level conditional entropy continues

declining with very long context in large LLMs. These works primarily characterize aggregate entropy or surprisal rather than context-gain trajectories for individual target characters.

Punctuation and context in neural models. Razzhigaev et al. (2025) showed punctuation carries disproportionate contextual information in Transformer attention. Chauhan et al. (2026) demonstrated model-dependent computational roles for punctuation. Our work measures context dependency at the raw character level, using only count-based n-gram models without neural model involvement.

Code naturalness and localness. Hindle et al. (2012) established that source code is highly predictable under n-gram models. Tu et al. (2014) showed local cache statistics account for much of this predictability — and that syntax tokens (separators, brackets) are particularly predictable locally. Our work extends this: not just *how predictable* but *how much does additional context reduce surprisal* at each position, and whether that marginal gain differs between syntax and identifier characters.

Surprisal reduction curves. Measuring how surprisal or entropy changes as context length grows has a long history (Shannon, 1951; Brown et al., 1992) and continues in recent large-model work (Scheibner et al., 2025). Studying surprisal reduction as a function of context length — rather than reporting a single aggregate perplexity — is therefore established methodology, not a novel contribution of the present work.

Variable-order Markov models and adaptive context. Variable-order Markov models (VOM; Ron et al., 1996) and the related Prediction by Partial Matching (PPM) algorithm (Cleary & Witten, 1984; Bell et al., 1990) address a question superficially similar to this paper: how much context should be used when predicting a character? VOM and PPM answer by selecting context length *conditioned on the observed history*: use the longest available context that has sufficient frequency statistics, backing off to shorter contexts when the long history was unseen in training. Context length is therefore a function of what appears *before* the prediction target. The present work asks a complementary but distinct question: conditioned on the *target character being x*, how much does context of length k reduce x’s surprisal on average, across all corpus positions where x occurs? This is **target-conditioned context benefit** rather than history-conditioned context selection — the independent variable is the identity of the character being predicted, not the composition of the preceding history. VOM addresses *when to use more context* (a history-dependent decision made at each position); CG_x(k) addresses *which targets benefit more from more context* (a target-dependent measurement aggregated over all positions). These are orthogonal decompositions of the same prediction problem: a VOM model can decide to use k=5 at a given position without revealing whether the benefit of that longer context is distributed uniformly across all possible next characters or concentrated in structural ones.

Gap. The present work differs in three ways not found together in prior work: (1) decomposition at the level of individual characters rather than word categories, (2) a frequency-controlled regression comparing structural vs. lexical character types across prose and source code, and (3) Python tokenization stratification to separate syntax operators from identical characters in strings and comments. CG_x(k; D) is defined for this study as an operationalization of these measurements; the underlying operation of subtracting surprisal estimates is not claimed as a new mathematical quantity.

3. Methodology

3.1 Notation and terminology

Symbol	Definition
x	Individual character (e.g., , , a, ()
D	Corpus; D_train / D_test non-overlapping splits
k	Context length (number of preceding characters)
S_x(k; D)	Target-character mean surprisal of x at context k in D
CG_x(k; D)	Context gain = S_x(1; D) – S_x(k; D)
k_peak(x; D)	argmax CG_x(k; D) within reliable k range

Symbol	Definition
Coverage _x (k; D)	Fraction of test positions for x where k-char context was seen in training

Notation note. Surprisal is the negative log-probability assigned to a particular observed outcome. Entropy is the probability-weighted expected surprisal over the distribution of possible outcomes. In this study, $S_x(k; D)$ averages surprisal only over positions where the observed target character is x — not over the full distribution of next characters. We therefore write S rather than H to make explicit that this is a per-target-outcome average, not an entropy.

3.2 Target-character mean surprisal

For a Laplace-smoothed n-gram trained on D_{train} with vocabulary size $|V|$:

$$P(x_t \mid x_{\{t-k:t-1\}}) = (\text{count}(x_{\{t-k:t-1\}}, x_t) + 1) / (\text{count}(x_{\{t-k:t-1\}}) + |V|)$$

$$S_x(k; D) = E[-\log_2 P(x_t \mid x_{\{t-k:t-1\}}) \mid x_t = x]$$

= mean over test positions where $x_t = x$

Count-based n-gram models with Laplace smoothing serve as a conservative, interpretable baseline. The context-gain values reported here are model-specific estimates obtained without learned representations or long-range neural capacity, providing a reproducible reference point for future comparisons against neural character-level models.

3.3 Per-symbol analysis: individual characters, then classification

We measure $S_x(k; D)$ and $CG_x(k; D)$ for **every individual character** x in the corpus vocabulary. We then classify characters as structural or lexical *after* measurement. This avoids the apples-to-oranges problem of comparing singleton punctuation characters (, , .) against broad aggregates (all vowels, all consonants).

Natural language classification: - Structural: , . ; : ? ! ' " () [] { } - Lexical: alphabetic characters (a-z, A-Z), digits, space - Ambiguous: \n (excluded from primary comparison)

Source code classification (Code1): - Structural: Python syntax operators () [] { } , : ; = . @ + - * / % & | ^ ~ < > @ — *only* when the character appears in Python's OP token category, as determined by the tokenizer - Lexical: alphabetic and digit characters in NAME or NUMBER tokens - Excluded from primary comparison: characters inside STRING or COMMENT tokens; NEWLINE/INDENT/DEDENT whitespace (ambiguous)

This means ? and ! in Python are excluded from the structural set entirely — they are not Python syntax operators and appear almost exclusively inside strings and comments.

3.4 Python tokenization

We apply Python's tokenize module to each .py file individually before concatenation, assigning each character position a stratum: syntax, keyword, identifier, string, comment, numeric, whitespace, other. Only syntax-stratum characters are candidates for the structural category in Code1.

Stratum distribution in Code1 test set: - string: 33.8% - identifier: 25.3% - other: 13.7% - comment: 8.6% - whitespace: 8.1% - syntax: 5.4% - keyword: 4.6% - numeric: 0.4%

In this corpus, 42% of characters fall inside STRING or COMMENT tokens — a corpus-specific figure that reflects the stdlib's heavy use of docstrings and string constants, not a general property of Python codebases. Strings may contain paths, regex, serialised data, SQL, or program constants — not necessarily natural language. When interpreting punctuation characters specifically as executable syntax operators, tokenizer stratification is required to separate them from identical characters appearing in string literals and comments.

3.5 Coverage diagnostic and reliable k range

At high k , most test contexts were never seen in training. Laplace smoothing then produces loss near $-\log_2(1/|V|)$ regardless of the true dependency — a measurement artifact, not a signal.

$$\text{Coverage}_x(k; D) = \frac{(\text{test positions for } x \text{ where } k\text{-char context was in } D_{\text{train}})}{(\text{total test positions for } x)}$$

Global Coverage($k; D$) = same but over all characters

We report the **reliable k range** as k where global Coverage($k; D$) $\geq 50\%$. We additionally enforce per-symbol coverage $\geq 50\%$ for each (character, k) observation in the regression. Sensitivity to thresholds of 25%, 50%, and 75% is reported in Section 4.1. The reliable range is corpus-specific:

Corpus	Reliable k range	k=8 coverage
NL1 tinyshakespeare	$k \leq 7$	43.9% — sparse
NL2 Pride & Prejudice	$k \leq 8$	61.6%
Code1 Python stdlib	$k \leq 10$	63.1%

NL2 stays reliable at $k=8$ despite being the smallest corpus. NL1 breaks at $k=8$ despite being larger. This is an observed coverage difference between the two corpora; the mechanism — whether it reflects authorial consistency, genre, or something else — is not directly measured here and should not be interpreted as a claim about Austen's prose style.

3.6 Confidence intervals

For $n \geq 30$ test instances: 95% CI on mean surprisal using the CLT ($\text{mean} \pm 1.96 \times \text{SE}$). For $n < 30$: bootstrap 95% CI (1,000 resamples). All symbols with $n < 30$ are excluded from primary analysis and flagged if reported.

CG_peak CI. Since $\text{CG}_x(k) = S_x(1) - S_x(k)$, the 95% CI on CG is computed as a conservative bound: $\text{CI}_{\text{lo}} = S_x(1)_{\text{lo}} - S_x(k_{\text{peak}})_{\text{hi}}$ and $\text{CI}_{\text{hi}} = S_x(1)_{\text{hi}} - S_x(k_{\text{peak}})_{\text{lo}}$. This treats the two surprisal estimates as if they were independent, producing CIs that are wider than necessary. Additionally, $\text{CG}_{\text{peak}} = \max_k \text{CG}_x(k)$ is a selected maximum from noisy estimates, introducing selection optimism: $E[\max(\text{CG}_{\text{hat}})] > \max(E[\text{CG}_{\text{hat}}])$. Characters with more eligible k values have more chances to record an extreme maximum. These CG_{peak} CIs should therefore be interpreted as approximate and likely anti-conservative for the peak estimate specifically. The regression analysis (Section 4.1), which uses each k -specific CG observation rather than a selected maximum, is not affected by this selection issue.

3.7 Statistical tests

Two tests answer two different questions. Mann-Whitney asks whether structural characters have higher Mean Context Gain than lexical characters — a comparison of average magnitude over the reliable context range. A separate peak comparison is retained only as an exploratory reference. The regression asks whether structural characters accumulate context gain faster per unit of additional context — a comparison of slopes. Both are reported throughout the paper and are not interchangeable; they can legitimately give different answers for the same corpus.

Mann-Whitney U: one-sided test (structural $>$ lexical) on per-symbol Mean Context Gain distributions. Tests whether structural characters have stochastically higher Mean Context Gain than lexical characters. A secondary Mann-Whitney on CG_{peak} is reported as an exploratory reference; CG_{peak} results are labelled exploratory throughout.

Regression: for each corpus, pooling all (x, k) observations for $k \in \{2, 3, \dots, k_{\text{max}}\}$ within the reliable range:

$$\text{CG}_x(k; D) = \beta_0 + \beta_1 \log_2(k) + \beta_2 \text{Structural}_x + \beta_3 [\log_2(k) \times \text{Structural}_x] + \beta_4 \log_2(\text{Freq}_x) + \epsilon$$

Note: $k=1$ serves as the baseline for computing $CG_x(k) = S_x(1; D) - S_x(k; D)$ but is **not itself a regression observation**. By definition $CG_x(1) = 0$ for every character; including it would constrain the fit at a deterministically zero outcome. The intercept and β_2 therefore represent extrapolated values at $\log_2(k)=0$ (i.e., $k=1$), outside the regression's observed range. The key coefficient β_3 is unaffected by this centering choice.

The key coefficient is β_3 : the additional context-gain rate for structural symbols per unit $\log_2(k)$, after controlling for frequency and baseline context growth. $\beta_3 > 0$ means structural symbols benefit more per doubling of context, after controlling for log character frequency.

Analysis hierarchy. Primary inferential claim: β_3 in NL2 (Pride & Prejudice), from the regression above, validated by permutation test and all robustness checks. Secondary/directional: β_3 in NL1 (Shakespeare), which is positive but does not survive permutation. Cross-domain: the exact permutation test over all $C(8,5)=56$ corpus assignments (Section 4.8). Robustness checks (exploratory): LOO regression, KT smoothing, coverage threshold sweep, split robustness, classification sensitivity — all reported in Sections 4.6 and 4.8. Exploratory observations: Mann-Whitney peak-gain comparisons, Phase 2 directional results.

3.7.1 Trajectory and magnitude — formal definitions

Context dependence has two distinct components captured by different statistics; both appear in the results and it is important not to conflate them.

Trajectory (β_3): the differential slope of CG accumulation with $\log_2(k)$ for structural vs. lexical characters. Formally, β_3 is the coefficient on the $\log_2(k) \times \text{Structural}$ interaction in the regression above. A positive β_3 means structural characters gain predictive benefit at a *faster rate per doubling of context length* than lexical characters of comparable frequency. This component is estimated from the full (character, k) panel using OLS with cluster-robust standard errors.

Mean Context Gain (MCG): $\text{mean}\{CG_x(k; D) : k \text{ in reliable range}\}$. This is the mean of CG over the reliable k range per character. It averages over all reliable (character, k) observations and avoids the additional selection optimism introduced by taking the maximum over noisy k-specific estimates. It measures *how much* gain is accumulated on average across the reliable context range. $CG_peak = \max\{CG_x(k; D) : k \text{ in reliable range}\}$ is an alternative exploratory statistic: because it is a maximum over noisy estimates, $E[\max(CG_hat)] > \max(E[CG_hat])$ — it introduces selection optimism relative to the true maximum (see Limitation 4). Results using CG_peak are labelled exploratory throughout this paper.

Why these can dissociate. A character can have high Mean Context Gain but flat trajectory ($\beta_3 \approx 0$) if it achieves most of its gain early and then plateaus. Conversely, high trajectory with moderate Mean Context Gain would mean the character keeps benefiting from longer context but from a lower base. The regression measures trajectory; Mann-Whitney on per-symbol Mean Context Gain is the secondary magnitude comparison. In the regression, β_2 and β_3 are partially decoupled by construction: β_2 captures the level difference at the reference context length, while β_3 captures only the additional slope of CG growth per unit $\log_2(k)$. A character type can therefore have $\beta_2 > 0$ (higher absolute gain) and $\beta_3 \approx 0$ (same rate), or vice versa. The two statistics answer different questions and can legitimately give different answers for the same corpus.

3.8 Corpora

ID	Corpus	Domain	Size	Char vocab
NL1	tinyshakespeare	Natural language	~1.1M chars	65
NL2	Pride and Prejudice	Natural language	~694K chars	87
Code1	Python 3.12 stdlib (163 files)	Source code	~4.6M chars	164

Phase 2 replication adds three natural-language corpora and two source-code corpora beyond the three canonical corpora; all eight are described in Section 4.7.

All splits: 80% train / 10% test on raw characters. (A 10% validation split was created but not used in any reported computation; all surprisal estimates and regression observations are from the held-out test portion only.) Candidate context lengths were evaluated up to the per-corpus $k_{\max}$ determined by the coverage probe (Section 3.5); $k=1$ serves as the CG baseline; regression observations use $k \in \{2, \dots, k_{\max}\}$. The probe yielded $k_{\max} = 7$ for NL1, $k_{\max} = 8$ for NL2, and $k_{\max} = 10$ for Code1.

4. Results

The strongest single-corpus result is in Pride and Prejudice (NL2): structural characters have a significantly steeper context-gain trajectory than lexical characters ($\beta_3=+1.131$, $p=0.002$), confirmed by permutation test ($p=0.001$) and stable across coverage thresholds, functional forms, text splits (all 5 positive, 4/5 $p<0.05$), and lexical definition (alpha-only: $\Delta\beta_3=2.4\%$). Shakespeare (NL1) shows the same direction but does not survive the permutation test. Python source code (Code1) shows a null trajectory effect but an exploratory Mean Context Gain difference. An exact permutation over all $C(8,5)=56$ domain assignments of the eight Phase 2 corpora confirms that the observed NL/code partition achieves the maximum possible mean-difference ($p=1/56=0.018$). Evidence is reported in order: main regression (Section 4.1), magnitude comparison using Mean Context Gain (4.2), per-symbol profiles (4.3–4.5), robustness suite (4.6), replication across eight corpora (4.7), and the cross-domain permutation test (4.8).

4.1 Regression with cluster-robust standard errors

OLS with standard errors clustered by character (G = number of unique characters), enforcing per-symbol coverage $\geq 50\%$ for each (character, k) observation included.

Corpus	n obs	G (chars)	R ²	β_3	SE	95% CI	p
NL1 shakespeare	272	56	0.393	+0.549	0.250	[+0.047, +1.050]	0.033
NL2 pride_prej	310	45	0.701	+1.131	0.337	[+0.452, +1.810]	0.002
Code1 python	456	77	0.571	-0.024	0.278	[-0.577, +0.528]	0.930

Full coefficient table — NL1 (shakespeare):

Coefficient	β	SE	95% CI	p
intercept	-1.300	0.486	[-2.275, -0.326]	0.010
$\log_2(k)$	-0.242	0.145	[-0.533, +0.048]	0.101
Structural	-0.369	0.490	[-1.351, +0.614]	0.455
$\log_2(k) \times \text{Structural}$	+0.549	0.250	[+0.047, +1.050]	0.033
$\log_2(\text{Freq})$	-0.315	0.051	[-0.416, -0.213]	<0.001

Full coefficient table — NL2 (pride_prej):

Coefficient	β	SE	95% CI	p
intercept	-0.034	0.231	[-0.499, +0.432]	0.886
$\log_2(k)$	-0.856	0.094	[-1.046, -0.666]	<0.001
Structural	-2.086	0.667	[-3.430, -0.743]	0.003
$\log_2(k) \times \text{Structural}$	+1.131	0.337	[+0.452, +1.810]	0.002
$\log_2(\text{Freq})$	-0.378	0.024	[-0.427, -0.330]	<0.001

Full coefficient table — Code1 (python):

Coefficient	β	SE	95% CI	p
intercept	-2.315	0.551	[-3.413, -1.217]	<0.001

Coefficient	β	SE	95% CI	p
$\log_2(k)$	-0.133	0.153	[-0.438, +0.171]	0.386
Structural	+0.202	0.565	[-0.924, +1.327]	0.722
$\log_2(k) \times \text{Structural}$	-0.024	0.278	[-0.577, +0.528]	0.930
$\log_2(\text{Freq})$	-0.467	0.045	[-0.557, -0.377]	<0.001

The negative β_1 values reflect that high-frequency lexical characters — which dominate the panel — achieve most of their context gain at $k=2-3$ and then plateau or slightly reverse; the regression fits this average downward trend. This does not mean context is unhelpful: for structural characters in NL2 and NL1, the effective slope is $\beta_1 + \beta_3$. In NL2, $\beta_1 + \beta_3 = -0.856 + 1.131 = +0.275$ (positive), and in NL1, $-0.242 + 0.549 = +0.307$ (positive) — structural characters' CG continues to grow with $\log_2(k)$ within the reliable range, consistent with the observed k_{peak} values of 4–6 for most structural characters. For Code1, β_3 is null (-0.024 , $p=0.930$), so no differential slope interpretation applies; structural characters in Python reach their CG peak at $k=2-3$ at roughly the same rate as lexical characters. The negative β_2 (Structural main effect) in NL2 represents the structural-vs-lexical difference extrapolated at $\log_2(k)=0$, outside the regression's observed range; its sign should not be interpreted substantively. The meaningful coefficient is β_3 .

Robustness. NL2 is stable across coverage thresholds (β_3 range +0.982 to +1.142, all $p<0.01$), functional forms (positive and significant under linear- k ; all six categorical- k interaction terms positive and monotonically increasing), and a 10,000-permutation character-label shuffle (permutation $p=0.001$). NL1 is significant at $\tau=0.25$ and $\tau=0.50$ but not $\tau=0.75$, and fails the character-label permutation test ($p=0.093$); it is treated as directional throughout. Code1 is null under all specifications. Under Bonferroni correction for three simultaneous regression tests ($\alpha \approx 0.017$), NL2 survives ($p=0.002$); NL1 does not ($p=0.033$). Full robustness tables are in the Appendix.

4.2 Magnitude comparison: Mean Context Gain

Mean Context Gain (mean CG over the reliable k range per character, Mann-Whitney one-sided structural > lexical):

Corpus	Struct n	Lex n	Struct median MCG	Lex median MCG	p-value	Interpretation
NL1 shakespeare	7	49	1.563 bits	0.050 bits	0.006	nominally significant
NL2 pride_prej	5	40	1.703 bits	0.697 bits	0.040	nominally significant
Code1 python	20	57	1.740 bits	1.105 bits	0.019	exploratory: trajectory null in Code1
NL3 reuters	8	62	2.702 bits	1.143 bits	0.015	nominally significant

Mean Context Gain comparisons are secondary to the trajectory regression and are reported without correction for multiple comparisons; the regression with Bonferroni correction (Section 4.1) remains the primary inferential test. For transparency: under Bonferroni correction across the four reported Mean Context Gain comparisons ($\alpha=0.05/4=0.0125$), only NL1 remains significant ($p=0.006$); NL2, Reuters, and Code1 are nominally significant only. Mean Context Gain is the primary magnitude statistic here because it averages over all reliable k values rather than selecting the maximum, avoiding the selection-optimism bias that inflates CG_{peak} ($E[\max(\text{CG}_{\text{hat}})] > \max(E[\text{CG}_{\text{hat}}])$). In NL2, Mean Context Gain $p=0.040$ is nominally significant — Mean Context Gain is more stable than CG_{peak} at $n=5$ structural characters. In the three reported NL Mean Context Gain comparisons (NL1, NL2, Reuters), structural characters have higher Mean Context Gain; all three reach nominal significance. In Code1, Mean Context Gain $p=0.019$ is exploratory: structural chars in Python achieve higher mean CG than lexical chars, but the trajectory is null ($\beta_3=-0.024$), so this magnitude observation is a contrast-case finding only.

Framework utility: per-character saturation profiles. The per-character Mean Context Gain output illustrates the framework's practical value. In Pride and Prejudice, ! sustains mean CG=2.97 bits over $k=2..8$ — the highest of any

character — accumulating predictive benefit at each additional context unit. , achieves mean CG=0.65 bits and saturates earlier ($k_{\text{peak}}=3$ in the profile table). A system allocating context budget across prediction positions could use this per-character saturation profile to preferentially extend context for !-type positions in prose, where additional context remains measurably informative, while truncating earlier for positions where common high-frequency characters are expected. These are n-gram-based measurements; whether analogous saturation patterns hold under neural estimators is an open empirical question, but the framework provides a concrete, reproducible baseline for such comparisons.

4.3 Selected per-symbol results — NL1 (tinyshakespeare)

Structural characters with $n \geq 30$:

Char	$S_x(1)$	CG_peak	95% CI	k_{peak}	sym_cov
!	8.225	2.481	[2.27, 2.70]	6	79.8%
?	8.027	2.183	[1.99, 2.38]	6	83.1%
'	7.149	1.880	[1.68, 2.08]	4	95.2%
;	7.495	1.810	[1.61, 2.01]	5	90.9%
.	6.336	1.347	[1.21, 1.48]	4	96.5%
,	4.957	1.183	[1.08, 1.29]	3	99.6%
:	4.651	0.820	[0.56, 1.08]	2	99.9%

Comparable high-frequency lexical characters:

Char	$S_x(1)$	CG_peak	95% CI	k_{peak}	sym_cov
v	5.707	2.536	[2.29, 2.79]	3	99.9%
g	4.813	1.314	[1.11, 1.52]	3	99.9%
o	3.444	0.933	[0.86, 1.01]	2	100%
e	2.537	0.296	[0.22, 0.38]	3	98.9%

Rare lexical characters (v, g) can exceed common structural characters in raw context gain; the regression controls for this frequency effect, making β_3 the primary inferential comparison. CIs shown are subject to selection optimism at the peak k ; see Section 3.6.

4.4 Python Code1 — structural chars (syntax stratum only)

Key structural characters with $n \geq 30$ in syntax stratum:

Char	$S_x(1)$	CG_peak	k_{peak}	sym_cov
&	12.35	5.73	6	56.1%
%	9.05	5.60	2	100%
@	11.34	5.26	7	90.8%
{	9.78	3.35	3	98.3%
+	9.16	3.03	3	94.6%
[	8.75	2.48	4	91.4%
=	5.99	1.93	3	98.3%
(	5.77	1.67	3	97.2%
.	4.31	1.54	3	96.7%
:	4.40	0.78	3	98.5%
,	5.01	0.65	2	99.9%

Char S_x(1) CG_peak k_peak sym_cov
) 5.16 0.55 2 99.9%

: and , in Python have *lower* context gain than in Shakespeare — consistent with grammar-enforced placement making them more locally predictable. &, %, @ have very high CG but low coverage at the peak k (56-90%) — treat with caution.

4.5 The colon cross-corpus comparison

The colon : appears in both Shakespeare (following speaker names, e.g. HAMLET:) and Python (following keywords, e.g. if x:). The Python colon — despite being grammar-enforced — has slightly lower context gain than the Shakespeare colon, consistent with short repeated keyword patterns making it locally predictable without requiring long-range context.

Corpus	S_x(1)	CG_peak	k_peak	n_test
NL1 shakespeare	4.651	0.820	2	1272
Code1 python (syntax only)	4.400	0.781	3	2816

Python colon (syntax stratum, n=2816) has *slightly lower* context gain than Shakespeare colon — the opposite of what a naive grammar-complexity argument would suggest. This is consistent with the hypothesis that grammar-enforced placement after short keywords (if, def, for) makes the 2–3 character context highly informative; however, preceding-context distributions were not directly measured. Shakespeare colons follow varied speaker names: the bigram provides some signal, but the 2-char window is already saturated.

This is a methodologically important result: grammar enforcement does not necessarily produce *longer* context horizons at the character level. The result is consistent with short repeated syntax patterns making some structural symbols more locally predictable than discourse-governed usage.

4.6 Robustness suite

The NL2 finding survives all kill tests: LOO regression (all five structural-character exclusions yield $\beta_3 > +0.90$, $p < 0.013$; all 20 Code1 exclusions null, $p > 0.44$); KT smoothing ($\beta_3=+1.114$, $\Delta=0.017$, $p=0.002$); five text splits (β_3 range +0.776 to +1.244, four of five $p<0.05$); and α -only lexical definition (shift $\leq 3.8\%$). Full tables are in the Appendix.

4.7 Replication across corpora

Phase 2 pipeline. All five NL corpora use the same character classification as the canonical analysis. All three code corpora use character-identity-only classification (no per-language tokenizer strata) for cross-language comparability; they should be interpreted as coarse-classification sensitivity analyses. The canonical Code1 result ($\beta_3=-0.024$, $p=0.930$), which uses full Python tokenizer stratification, remains the primary code finding.

Phase 2 corpora:

ID	Corpus	Domain	Size	Source
NL1	tinyshakespeare	Drama	~1.1M chars	canonical
NL2	Pride and Prejudice	19c novel	~694K chars	canonical
NL3	Reuters-21578	News wire	~9.0M chars	NLTK
NL4	Bible KJV	Religious prose	~4.6M chars	Project Gutenberg
NL5	WikiText-103	Wikipedia	~2.0M chars	HuggingFace
Code1	Python 3.12 stdlib	Source code	~4.6M chars	canonical
Code2	Node.js v22 stdlib	Source code	~3.3M chars	GitHub

ID	Corpus	Domain	Size	Source
Code3	Apache Commons Lang	Source code	~8.7M chars	GitHub

Consolidated results (β_3 , cluster-robust OLS):

Corpus	Type	β_3	SE	95% CI	p	n obs
Pride & Prejudice	NL	+1.131	0.337	[+0.452, +1.810]	0.002	310
Reuters	NL	+0.712	0.348	[+0.018, +1.405]	0.044	490
Bible KJV	NL	+0.693	0.391	[−0.087, +1.474]	0.081	440
WikiText-103	NL	+0.676	0.404	[−0.131, +1.482]	0.099	470
Shakespeare	NL	+0.549	0.250	[+0.047, +1.050]	0.033	272
Python stdlib	Code	−0.024	0.278	[−0.577, +0.528]	0.930	456
Node.js stdlib†	Code	−0.321	0.192	[−0.703, +0.060]	0.098	547
Commons Lang (Java)	Code	+0.075	0.264	[−0.451, +0.601]	0.777	556

Figure 1 (forest_plot.png): β_3 estimates with 95% CIs across all eight corpora. Diamonds: $p < 0.05$; circles: $p \geq 0.05$. NL panel in blue (left), code panel in red-orange (right). Node.js shows unstratified Phase 2 result; see Appendix for approximate stratification check.

Reuters — second nominally significant cross-corpus result. Reuters reaches nominal significance (β_3 =+0.712, p =0.044), confirmed by LOO regression (all 8 structural character exclusions positive; Appendix) and nominally significant Mean Context Gain (p =0.015).

Phase 2 code corpora use character-identity classification (no per-language tokenizer strata); Node.js stratification results are in the Appendix.

Results. All five NL corpora show positive β_3 (range +0.55 to +1.13). Three have nominal cluster-robust $p < 0.05$ (Pride & Prejudice, Reuters, and Shakespeare); the remaining two are positive but not individually significant. Under Bonferroni correction across the eight corpus-level tests, only NL2 remains significant (p =0.002). All three code corpora show β_3 near zero or slightly negative; none is significant.

Statistical note. Under Bonferroni correction for eight simultaneous tests, $\alpha \approx 0.006$; only NL2 survives (p =0.002). The replication strength is therefore directional across the NL panel, not each-corpus-significant. A one-sided sign test over the five NL results — five positives out of five — yields $p = (0.5)^5 = 0.031$ under the null that each corpus’s β_3 sign is random, providing a conservative non-parametric replication summary. The formal joint test — whether the NL/code partition achieves a systematically larger mean-difference than chance — is the exact domain permutation test in Section 4.8 (p =1/56=0.018).

4.8 Cross-domain permutation test

An exact permutation test over all $C(8,5)$ =56 assignments of the eight Phase 2 corpora shows the observed NL/code partition achieves the maximum mean β_3 separation among all possible assignments (T_{obs} =+0.842, exact p =1/56=0.018).

5. Discussion

5.1 Main finding

The NL2 structural-trajectory result is robust across coverage thresholds, functional forms, five text splits, and lexical-definition variants; Reuters is confirmed positive under leave-one-character-out analysis. In Pride and

Prejudice, structural symbols have a robustly steeper trajectory than lexical symbols after controlling for character frequency ($\beta_3=+1.131$, cluster-robust $p=0.002$, permutation $p=0.001$), with Mean Context Gain nominally significant ($p=0.040$). Reuters provides a second nominally significant cross-corpus result ($\beta_3=+0.712$, $p=0.044$; Mean Context Gain $p=0.015$). Shakespeare is directionally consistent ($\beta_3=+0.549$, $p=0.033$) but does not survive the character-label permutation test ($p=0.093$); it is purely directional, not independent replication. Python source code serves as a domain contrast case with no significant trajectory effect ($\beta_3=-0.024$, $p=0.930$).

5.2 Trajectory and Mean Context Gain as distinct estimands

The framework decomposes context dependence into two quantities that need not move together. Trajectory (β_3) measures the rate at which predictive benefit accumulates per unit log-context — how steeply the gain curve rises. Mean Context Gain (mean CG over the reliable range) measures the average gain across the reliable context range. A character type can gain rapidly but saturate early, or gain more slowly to a higher asymptote; trajectory and Mean Context Gain can separate. The structural-trajectory effect in NL prose is a trajectory finding: structural characters accumulate predictive benefit faster than lexical characters after controlling for frequency.

Mean Context Gain is the primary magnitude metric throughout this paper because it avoids the additional selection optimism introduced by taking the maximum over noisy k -specific estimates. $CG_peak = \max_k CG_x(k)$ introduces that selection optimism ($E[\max(CG_hat)] > \max(E[CG_hat])$) and appears only as an exploratory reference in this paper. The CG values are estimator-dependent: they describe what a Laplace-smoothed n -gram finds in these corpora. No guaranteed relationship holds between n -gram CG and Transformer CG; the n -gram framework was chosen for transparency and reproducibility — it does not conflate model capacity with corpus properties, and any replication can use identical count statistics. Every measurement is corpus-conditional: the reliable k range and CG values must be recomputed per corpus and cannot be inferred from corpus size.

5.3 Implications for context engineering

The trajectory/magnitude framework has a direct practical analogue: different prediction targets saturate at different context lengths, and the structural-versus-lexical pattern varies across corpora. Structural positions in natural-language prose continue to benefit from context beyond what saturates lexical characters. If analogous target-specific saturation patterns hold in neural models, retrieval or compression systems that treat all positions uniformly could leave predictive gain unused. This moves the design question from *how much context can the model accept?* toward *how much context does this specific prediction target benefit from?*

The per-character Mean Context Gain profiles illustrate this concretely in *Pride and Prejudice*: ! achieves mean CG=2.97 bits over $k=2..8$, accumulating benefit through $k=6$; , achieves 0.65 bits and saturates earlier; e, h, a show near-zero or slightly negative mean CG. A context-budget system could preferentially extend context for !-type positions while truncating earlier at e-type positions, where additional context provides no measured benefit. We make no claim that n -gram saturation orderings transfer directly to neural models; this is a reproducible baseline that can be compared against neural measurements at the same character level. A natural follow-on question is whether these context-dependency differences are preserved or altered when characters are absorbed into subword tokens by BPE merging — testable using $S_x(k; D)$ as a character-level baseline.

5.4 Limitations

1. **Alternative smoothing:** Laplace smoothing is suboptimal at high k . Per-symbol coverage filtering ($\geq 50\%$) and frequency control (β_4) address the primary Laplace bias mechanisms, and the coverage sensitivity analysis (Section 4.1) shows the NL2 finding is robust across all three thresholds. A true interpolated Kneser-Ney test would require recursive backoff ($k \rightarrow k-1 \rightarrow \dots \rightarrow$ unigram), which is beyond the scope of this study.
2. **NL1 directional, not robust:** The NL1 β_3 finding is positive and cluster-robust at $\tau=0.50$ ($p=0.033$) but does not survive the character-label permutation test ($p=0.093$, Section 4.1). With only $G=7$ structural character clusters, the cluster-robust t -approximation is optimistic. NL1 should be read as directionally consistent with NL2, not as independent replication.

3. **NL2 power:** Only 5 structural characters cleared $n \geq 30$ in *Pride and Prejudice*, making Mann-Whitney underpowered for the Mean Context Gain test ($p=0.040$ is borderline significant at $n=5$). The trajectory regression is validated by the permutation test ($p=0.001$): despite only 5 structural clusters, the observed $\beta_3=+1.131$ is in the top 0.1% of the permutation null distribution.
4. **CG_peak selection optimism:** $CG_peak = \max_k CG_x(k)$ is upward-biased ($E[\max(CG_hat)] > \max(E[CG_hat])$). The primary magnitude metric throughout this paper is Mean Context Gain (mean CG over the reliable range), which avoids this additional selection optimism. CG_peak is reported as an exploratory reference only; the Code1 Mean Context Gain exploratory result ($p=0.019$) is the relevant magnitude signal for the code contrast case, not the CG_peak comparison.
5. **k range and corpus scale:** The reliable range is corpus-specific: $k \leq 7$ for NL1, $k \leq 8$ for NL2, and $k \leq 10$ for Code1, determined by the per-symbol coverage probe (Section 3.5). For corpora exceeding $\sim 100M$ characters, the in-memory n-gram frequency tables used here become memory-prohibitive; KenLM (Heafield, 2011) is the natural replacement, building a compressed trie under a fixed memory budget — though it uses Modified Kneser-Ney smoothing, so CG values would not be numerically identical to those reported here. A neural character-level model could further extend the reliable range beyond $k = 10$.
6. **Regression random effects:** The regression uses cluster-robust standard errors (clustered by character), which accounts for within-character correlation across k values. A character random-intercept model would additionally capture character-level variance in baseline surprisal.
7. **Classification sensitivity:** The natural-language comparison treats alphabetic characters, digits, and space as lexical, while punctuation and delimiters are structural. We test the alpha-only alternative (excluding digits and space from the lexical category) in Section 4.6.4; β_3 shifts by $\leq 3.8\%$ in both NL corpora and significance is retained. Further alternative schemes — for example, treating space as its own category, or varying the structural set — remain untested.

6. Conclusion

We introduced a per-character framework that decomposes context dependence into trajectory and Mean Context Gain. The strongest evidence is a structural-trajectory effect in *Pride and Prejudice*, with Reuters providing a second nominally significant cross-corpus result; code corpora serve as contrast cases with no significant trajectory effect. The framework exposes target-specific context sensitivity invisible to aggregate perplexity and provides a reproducible baseline for future adaptive-context work.

References

- Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*.
- Shannon, C. E. (1951). Prediction and entropy of printed English. *Bell System Technical Journal*.
- Brown, P. F., et al. (1992). An estimate of an upper bound for the entropy of English. *Computational Linguistics*.
- Cleary, J. G., & Witten, I. H. (1984). Data compression using adaptive coding and partial string matching. *IEEE Transactions on Communications*, 32(4), 396–402.
- Bell, T. C., Cleary, J. G., & Witten, I. H. (1990). *Text Compression*. Prentice-Hall.
- Ron, D., Singer, Y., & Tishby, N. (1996). The power of amnesia: Learning probabilistic automata with variable memory length. *Machine Learning*, 25(2–3), 117–149.
- Hindle, A., et al. (2012). On the naturalness of software. *ICSE 2012*.
- Tu, Z., Su, Z., & Devanbu, P. (2014). On the localness of software. *FSE 2014*.
- Scheibner, C., Smith, L. M., & Bialek, W. (2025). Large language models and the entropy of English. *arXiv:2512.24969*.

Appendix — Reproducibility

Code: [Anonymous]

Node.js approximate stratification (robustness check for Section 4.7). Phase 2 code corpora use character-identity classification without per-language tokenizer strata. For Node.js, we applied an approximate regex-based stratification to partially address this asymmetry: each corpus character is labelled code (45.9%), string literal (40.0%), or comment (14.1%). Characters with <50% code-stratum purity are excluded from the structural set (*, /, -, @, ^, ~, \). Re-running OLS on the stratified panel yields $\beta_3=+0.088$ (SE=0.175, 95% CI=[−0.256, +0.431], $p=0.618$, $n=537$, $G=79$), vs. unstratified $\beta_3=−0.321$ ($p=0.098$). Both are null. The stratified estimate is closer to zero, consistent with the weakly-negative unstratified result being partly attributable to contaminated characters (e.g., - as prose hyphen in string literals). Script: `tmlr_experiments/run_js_stratified.py`; output: `results_tmlr/nodejs_stratified_result.csv`.

Reuters LOO detail. LOO regression excluding each of the 8 Reuters structural characters one at a time: all 8 produce positive β_3 (range +0.564 to +0.945). Script: `tmlr_experiments/run_reuters_robustness.py`; output: `results_tmlr/reuters_loo.csv`.

Robustness tables (Sections 4.1 and 4.6).

Coverage threshold sensitivity (τ = minimum per-symbol coverage fraction; β_3 from cluster-robust OLS):

Corpus	$\tau=0.25$	$\tau=0.50$ (main)	$\tau=0.75$
NL1 shakespeare	+0.636**	+0.549*	+0.337 ($p=0.18$)
NL2 pride_prej	+1.142**	+1.131**	+0.982**
Code1 python	+0.008 (n.s.)	−0.024 (n.s.)	−0.059 (n.s.)

Character-label permutation test (10,000 permutations; structural/lexical labels shuffled within each corpus at fixed proportions):

Corpus	β_3	p cluster-robust	p permutation
NL1 shakespeare	+0.549	0.033	0.093
NL2 pride_prej	+1.131	0.002	0.001
Code1 python	−0.024	0.930	0.532

Functional form comparison ($M1=\log_2 k$, $M2=\text{linear } k$, $M3=\text{categorical } k$; $AIC = n \cdot \ln(RSS/n) + 2p$):

Corpus	Model	AIC	b_interaction	p_interaction
NL1	M1 $\log_2(k)$	−12.3	+0.549	0.033*
NL1	M2 linear k	−15.4	+0.207	0.033*
NL1	M3 categorical k	−13.8	+0.573 (mean)	—
NL2	M1 $\log_2(k)$	−186.6	+1.131	0.002**
NL2	M2 linear k	−219.8	+0.375	0.001**
NL2	M3 categorical k	−241.4	+1.398 (mean)	—
Code1	M1 $\log_2(k)$	+3.3	−0.024	0.930

Corpus	Model	AIC	b_interaction	p_interaction
Code1	M2 linear k	0.0	−0.001	0.988
Code1	M3 categorical k +5.5	−0.119 (mean)	—	—

NL2 M3 categorical k-specific interaction terms (all positive, monotonically increasing): k=3: +0.40, k=4: +0.86, k=5: +1.29, k=6: +1.67, k=7: +1.99, k=8: +2.19.

NL2 leave-one-structural-out (LOO) regression:

Excluded	β_3	95% CI	p
!	+0.913	[+0.242, +1.585]	0.009
,	+1.318	[+0.598, +2.037]	0.001
.	+1.343	[+0.662, +2.024]	<0.001
;	+1.124	[+0.287, +1.961]	0.010
?	+0.957	[+0.221, +1.692]	0.012

All 20 Code1 structural-character exclusions: $\beta_3 \in [-0.184, +0.066]$, all $p > 0.44$.

Krichevsky–Trofimov (KT) smoothing ($\alpha=0.5$) vs. Laplace add-1:

Corpus	Laplace β_3	KT β_3	$\Delta\beta_3$	p (KT)
NL1 shakespeare	+0.549	+0.510	−0.039	0.039
NL2 pride_prej	+1.131	+1.114	−0.017	0.002
Code1 python	−0.024	−0.011	+0.013	0.968

NL2 split robustness (five contiguous 80%/10% train/test splits at evenly-spaced text offsets):

Split	Offset	β_3	95% CI	p
1	0	+1.095	[+0.394, +1.795]	0.003
2	73K	+1.093	[+0.433, +1.753]	0.002
3	146K	+0.776	[−0.007, +1.559]	0.052
4	219K	+1.244	[+0.518, +1.969]	0.001
5	292K	+0.776	[+0.017, +1.536]	0.045

Classification sensitivity (α -only vs. canonical lexical definition):

Corpus	Definition	β_3	SE	p
NL1	Definition A (alpha+digit+space)	+0.739	0.226	0.002
NL1	Definition B (alpha only)	+0.711	0.225	0.003
NL2	Definition A (alpha+digit+space)	+1.095	0.348	0.003
NL2	Definition B (alpha only)	+1.069	0.348	0.004

Within-pipeline A→B shift $\leq 3.8\%$; significance unchanged. (Baseline values differ from canonical because this pipeline uses no k-max cap; the inferential comparison is the within-pipeline A→B shift.)

Exact pipeline for manuscript values (inputs → code → outputs):

Step	Command	Output
1	python run_experiment_v2.py	results_canonical_snapshot/panel_*.csv — regression panels for NL1/NL2

Step	Command	Output
2	python run_validations.py	results_canonical_snapshot/robustness_b3.csv — NL1/NL2 β_3 at all τ
3	python code1_coverage_sensitivity.py	results_canonical_snapshot/code1_coverage_sensitivity.csv — Code1 β_3
4	python tmlr_experiments/run_p0.py	results_tmlr/leave_one_structural_out.csv, results_tmlr/smoothing_robustness.csv — LOO and KT robustness
5–7	phase2_download_corpora.py, run_phase2.py, make_forest_plot.py	Phase 2 regression panels, Figure 1, consolidated results
8	python tmlr_experiments/run_domain_permutation.py	results_tmlr/domain_permutation.csv — permutation test
9–10	run_split_robustness.py, run_classification_sensitivity.py	Split and classification-sensitivity checks
11	python tmlr_experiments/run_reuters_robustness.py	results_tmlr/reuters_loo.csv — Reuters LOO
12	python tmlr_experiments/run_auc_magnitude.py	results_tmlr/auc_magnitude.csv — Mean Context Gain
13	python tmlr_experiments/run_js_stratified.py	results_tmlr/nodejs_stratified_result.csv — Node.js stratification

All canonical outputs are preserved in results_canonical_snapshot/. Seed: 42 | Smoothing: Laplace (add-1) | Min n: 30.

Standalone profiler. character_context_profile.py takes any corpus and produces per-character context-gain trajectories with coverage diagnostics (--python flag enables tokenizer stratification for source code). Dependencies: numpy only.